Journal of Software: Evolution and Process

WILEY

Journal of Software: Evolution and Process

RESEARCH ARTICLE - STANDARDS

PSCSelector: Appropriate Program Slicing Criteria Selection Based on Parameter-Efficient Lightweight LLM

Jianming Chang | Lulu Wang | Zaixing Zhang | Mei Han | Jiahao Ren | Bixin Li

Computer Science and Engineering School, Southeast University, Nanjing, Jiangsu, China

Correspondence: Lulu Wang (wanglulu@seu.edu.cn)

Received: 23 November 2024 | **Revised:** 24 November 2025 | **Accepted:** 26 February 2026

Keywords: pretrained model | program slicing | slicing criterion

ABSTRACT

Program slicing is widely used in downstream software engineering tasks, such as bug localization and code feature extraction in deep learning. For program slicing, existing research focuses on dependency analysis during slicing while lacking the selection of slicing criteria before slicing. As the starting point of dependency analysis, the slicing criterion directly determines the results of the program slicing application. However, existing criterion selection is only based on empirical judgment and lacks an effective slicing criterion selection method. In this paper, we propose PSCSelector, an appropriate Program Slicing Criterion Selection framework based on a popular lightweight LLM CodeBERT. PSCSelector first constructs the detailed substatement level system dependency graph to obtain the context of the slicing criterion. CodeBERT is then fine-tuned in a parameter-efficient way to jointly embed the slicing criterion context and the application texts to capture the appropriate criterion pattern. To validate the effectiveness of PSCSelector in downstream tasks, we choose bug localization as the example application and build the corresponding PSCSelector. In the experiment, we evaluate the effectiveness of PSCSelector in bug dataset Defects4J by calculating the number of searched bugs and the size of the search scope. The experimental results show that the PSCSelector-selected slicing criterion can effectively search 74% of bugs and reduce 69% of the original slice, which uses the bug-triggering statement as the slicing criterion.

1 | Introduction

Program slicing generates relevant code snippets based on user points of interest, called the slicing criterion [1]. Program slicing is essentially a graph reachability analysis containing two main steps: slicing criterion selection and dependency analysis. Current program slicing methods [2, 3] mainly focus on dependency graph construction and dependency analysis to improve the accuracy of program slicing. However, the studies focusing on slicing criterion selection are limited. In most applications of program slicing, the slicing lacks the selection of the slicing criterion. For example, Cao et al. [4] use all API calls and pointer variables as the slicing criteria to obtain the semantics of vulnerability code through program slicing; Alqadi et al. [5] use all program segments as the slicing criteria and detect the possibility that a slice contains a bug using slice-based metrics; In bug localization [6, 7], program slicing usually uses the bug-triggering statement [8] as the slicing criterion and generates code that affects the criterion to search for bugs. These program slicing criterion settings may contain the following three problems:

1) The dependencies associated with the slicing criterion are complex. Program slices from such slicing criteria are enormous and do not help the user narrow down the scope of the concern. **2)** The slicing criterion relies on experience. This criterion setting may select an inappropriate slicing criterion, resulting in the loss of expected code in the slice. **3)** The number of selected slicing criteria is abundant. In real projects, there are many function calls and API usage, generating abundant slices and degrading the program slice usage.

© 2026 John Wiley & Sons Ltd.

With an appropriate slicing selection, the number of criteria can be effectively limited, and the selected criterion will provide a more accurate program slice. This paper proposes a pretrained model-based approach to select the appropriate slicing criterion by covering the target code and having a smaller dependency range than the current criterion settings. Current research studies have shown that large language models can improve many software engineering tasks by effectively capturing the program's knowledge [9]. Moreover, CodeBERT [10], as one popular large language model, has been assessed for having excellent generalizability in tasks for both natural language and programming language [11]. Inspired by these researches, this work constructs the framework of slicing criterion selection using CodeBERT, focusing on program slicing application requirements and semantics of the criterion-related corpora.

Due to the lack of a dataset that can be directly used for training an appropriate slicing criterion selection model, and furthermore, considering program slicing is widely used in bug localization for narrowing down the bug search scope [12–14], this paper uses bug localization as an example application and constructs the labeled criterion dataset. For bug localization, the number of bugs and the distance from the bugs are two important factors [15, 16]. Therefore, this paper labels the criteria based on the two factors. At the same time, to improve the model's learning ability, we balance the labels by labeling an equal number of appropriate and inappropriate slicing criteria in real-world projects.

Based on the dataset, we further analyze the fine-granularity dependencies of the labeled slicing criterion at the five levels, including package, class, method, statement, and sub-statement levels. After generating the detailed criterion information, a pretrained model is fine-tuned with LoRA [17], a parameter-efficient fine-tuning method, to effectively learn the contextual semantics of the criteria-related corpora and the semantics of the corresponding slicing requirements. Finally, the trained model is used to select the appropriate slicing criteria.

In summary, this paper makes the following contributions:

- **Dataset**. This paper constructs a first labeled slicing criterion dataset, shared with the community to boost the future research on program slicing criterion selection. The dataset contains 1200 appropriate slicing criteria and 1200 inappropriate slicing criteria to facilitate research in learning slicing criterion information.
- **Framework**. This paper first constructs a framework for the automatic selection of slicing criteria named PSCSelector, which to the best of our knowledge is the first framework that selects appropriate slicing criteria via the lightweight LLM.
- **Evaluation**. This paper validates the effectiveness of PSCSelector with actual bug data. Experiments demonstrate that the approach in this paper can effectively select appropriate slicing criteria to help actual bug localization.

The rest of the paper is organized as follows. Section 2 presents the motivation. Section 3 provides the background of our approach. Section 4 introduces the dataset construction. Section 5 explains the technical details of our approach. Section 6 introduces the experiments. Section 7 analyzes the experiment results. Section 8 lists the threats to the validity. Section 8 reviews the related work. Section 10 concludes our work.

2 | Motivation

Figure 1a depicts the bug and their corresponding codes of the commit[1] in the CLI project. This bug triggers a ClassCastException during the `parseAndHelp` execution, which disrupts the expected validation process. The buggy function `validate` in `GroupImpl.java` implements the `validate` defined in `Option.java` through the interface `Group`, enabling its polymorphic invocation in `Parser.java`. To ensure clarity, the related codes have been simplified. The code shaded pink highlights the buggy codes (Lines 249, 250, 253, and 254), while the code shaded blue indicates the triggering statement of the bug. To locate the buggy statements from the triggering statement, eight related source files need to be examined, comprising a total of 2095 lines of code. Based on this example, we make the following observation.

The current slicing criteria configuration generates numerous irrelevant slices. Figure 1b presents a simplified system dependency graph for the code example, with irrelevant details omitted for clarity. When localizing bugs on the dependency graph, bug localization methods based on program slicing typically set the bug-triggering statement as the slicing criterion [18]. However, the slices generated by these methods often encompass all nodes and dependencies, as illustrated the left in Figure 1c, including extraneous scopes unrelated to the bug location, such as nodes in WriteableCommandLineImpl.java, WriteableCommandLine.java, and HelpFormatter.java.

Based on the observation, we propose a new framework, PSCSelector, for selecting appropriate slicing criteria. PSCSelector analyzes fine-grained dependencies between the appropriate slicing criterion and other code to obtain the criterion context. Furthermore, PSCSelector uses the parameter-efficient fine-tuned LLM to aggregate the code context of the slicing criterion with the application text to learn the appropriate slicing criterion.

For the best hope of our method, program slicing will exclude complex and redundant dependency from the PSCSelector-selected criterion. For example, the slice-related nodes from the PSCSelector-selected criterion in the right of Figure 1c effectively capture the bug-related code while excluding many irrelevant nodes, offering a stark contrast to the larger slice shown in the left of Figure 1c.

3 | Background

3.1 | System Dependency Graph

The system dependency graph [19] is an advanced version of the procedure dependency graph [20] and includes both intra-procedural and interprocedural dependencies. The intraprocedural dependency mainly consists of data and control

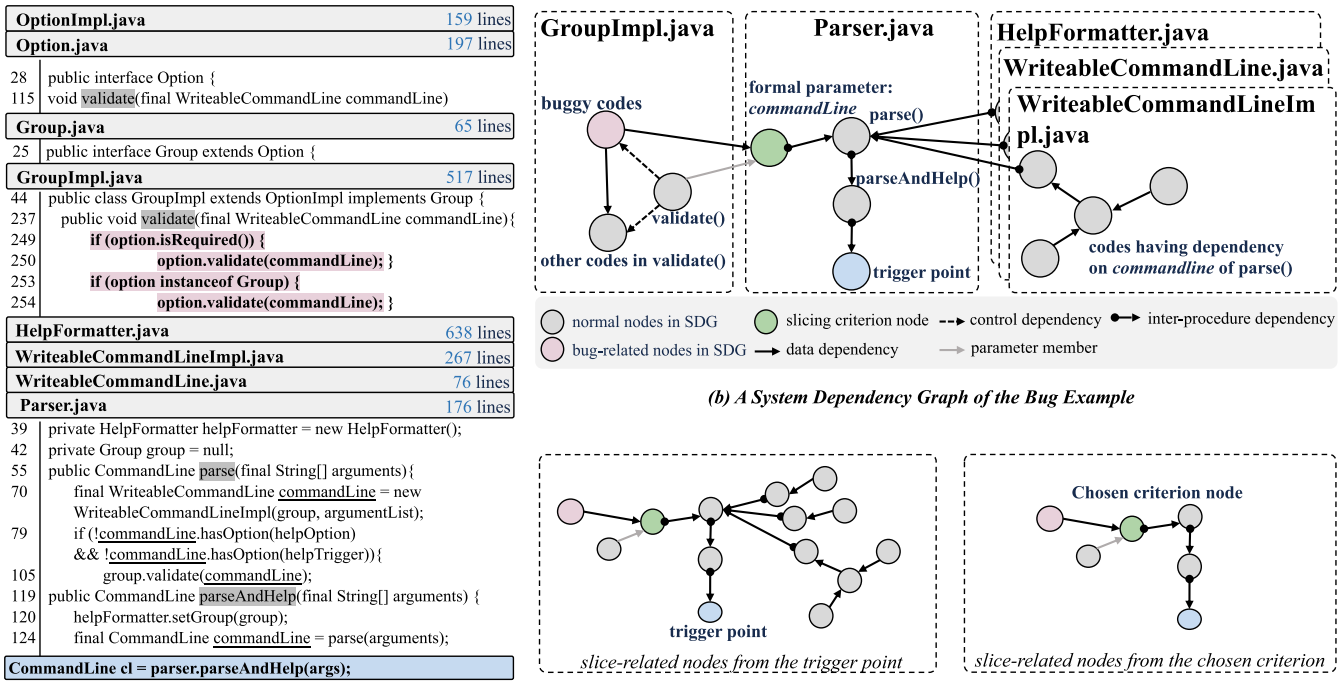

FIGURE 1 | The slice-related information of the bug example.

dependency. On the other hand, interprocedural dependencies are represented through "parameter In/Out" edges, which connect the formal parameters of a method with the actual parameters of the invocation. Specifically, "parameter in" connects the "actual in" node with the "formal in" node, while "parameter out" connects the "formal out" node with the "actual out" node. Additionally, when "actual in" depends on "actual out," a summary edge is constructed from "actual in" to "actual out." This summary edge reduces the traversal of the callee function and improves the slicing efficiency.

Based on current research, there are two prominent system dependency graphs that describe Java: JSysDG [21] and SSLDG [22]. Compared to JSysDG, SSLDG is more comprehensive and offers better support for polymorphic analysis by expanding type analysis [22]. SSLDG analyzes the member relationships between packages, classes, methods, statements, and substatement level elements, combining with intraprocedural and interprocedural dependencies. Additionally, it examines the object variable type at the substatement level. Composed of 17 types of edges and 16 types of nodes, SSLDG serves as the comprehensive intermediate representation in our approach, allowing for the semantic extraction of the slicing criterion.

3.2 | Program Slicing Criterion

The slicing criterion responds to the user's point of interest, defined as <*i, v*> [1], where *i* is the statement and *v* is the variable in the statement. The slicing criterion takes different forms as program slicing is applied to different applications. For example, in hierarchical slicing [23], the slicing criteria are tuples <*P(s), V*>, <*C(s), V*>, <*M(s), V*>, <*s, V*>, which correspond to the package, class, method, and statement levels, respectively; in chopping [24], the slicing criterion is a tuple<*s, t*>, where *s* is a source node and *t* is a target node; in type slicing [22], the slicing criterion is triple tuple<*s, v, t*>, where *s* is a statement, *v* is a subset of variables in *s*, and *t* is a possible type of *v*; in dynamic sling [25], the slicing criterion is triple <*input, i, V*>, which has one more program input case than static slicing criterion <*i, V*>.

Our approach learns the slicing criterion feature based on the detailed sub-statement level dependency graph. Therefore, the criterion in the training and selection phase is also defined as <*s, v, t*>, containing the appropriate statement, the variables, and variable types.

3.3 | Parameter-Efficient Tuning

LoRA is one of the state-of-the-art parameter-efficient tuning methods [26–28]. It works by introducing trainable low-rank matrices into the transformer layers to approximate weight updates. Instead of directly updating the pretrained matrix $W \in \mathbb{R}^{d\times k}$, LoRA adds two low-rank matrices, $W_{up} \in \mathbb{R}^{r\times k}$ and $W_{down} \in \mathbb{R}^{d\times r}$, to replace the W update. For a specific input X to the linear projection, LoRA modifies the projection output *Q* following:

$$Q = XW + XW' = XW + XW_{down}W_{up} \tag{1}$$

LoRA greatly reduces the number of parameters in the pretrained model. Compared to other popular parameter-efficient tuning methods, including Adapters and Prefix Tuning, LoRA's low-rank matrix can be merged with the original matrix without inference latency, improving training efficiency.

Therefore, in this paper, we use LoRA to fine-tune the pre-trained model.

4 | Dataset Construction

The dataset for this paper includes appropriate criteria for localizing bugs. To build the dataset, we followed prior program slicing studies [22, 29–31] and selected projects based on their GitHub star counts, reflecting project popularity and maturity. We searched for 14 popular open-source Java projects with the highest stars on GitHub, including "Dubbo," "antlr4," etc. These 14 projects were not part of the Defects4J dataset, which is used for evaluation. Table 1 summarizes the characteristics of these projects, including their star counts, the primary branch, the number of Java source files, and total lines of code. We searched bug-fixing commits in the 14 projects with the keywords "bug" and "fix". We collected 1200 bug-fixing commits and downloaded the corresponding original programs containing bugs, which are used to extract the appropriate slicing criteria. Since the commit message is a vital commit description that guides the bug localization, we also downloaded the messages of these commits.

With the information of 1200 bug-fixing commits, we compute the appropriate and inappropriate slicing criteria for each commit. The number of bugs and the distance from the bug are important metrics in bug localization [15, 16]. We calculate the appropriate slicing criterion based on the two critical factors: the distance between the buggy codes and the criterion and the number of buggy codes in the slice generated from the criterion. The calculation is shown in formula 2.

$$Score_c = Number_c + Distances_c \tag{2}$$

TABLE 1 | Java projects from GitHub with the most stars.

Project	Stars	Branch	Files	LOC
antlr4	18.5k	dev	410	44622
easyexcel	33.7k	master	325	23688
heritrix3	3.1k	master	446	64650
HikariCP	20.8k	dev	49	8286
java-design-patterns	98.3k	master	1235	69728
Mindustry	25.6k	master	802	133754
mybatis-3	20.3k	master	390	34667
paho.mqtt.java	2.2k	master	230	41271
presto	16.6k	master	6371	818208
teammates	1.8k	master	862	104708
dubbo	38.1k	3.0	2852	352564
hutool	24.7k	v5-master	1899	266424
jenkins	19.9k	master	1712	729822
retrofit	40.8k	master	290	41363

The $Number_c$ and $Distances_c$ describe the values of the two metrics, and the criterion c with the highest $Score_c$ is an appropriate criterion for program slicing as it generates a smaller slice to cover more buggy codes. Conversely, the criterion is inappropriate when the score is the lowest, indicating that distance between the criterion and buggy codes is the longest and the slice generated by the criterion contains the fewest buggy codes.

Specifically, the calculation consists of the following three steps:

Firstly, we construct the substatement level system dependency graphs for original programs of each commit.

Secondly, we locate the buggy nodes corresponding to the buggy codes in the graph and then calculate each node's distance to the buggy nodes in formula 3 and the number of buggy nodes searched by each node in formula 4:

$$Distances_n = \frac{\sum_{k=1}^{|B|} dis(n, b_k)}{\sum_{k=1}^{|B|} len(Slice_{forward}(b_k))} \tag{3}$$

$Distances_n$ represents the distance between the node and buggy nodes. In the formula, n denotes the node of SSLDG, b is one node of the buggy node set B, $dis(n,b)$ denotes the length of the dependency path between n and b in SSLDG, and $Slice_{forward}$(n) denotes the forward slice with n as the starting point. $len(Slice)$ denotes the sum of all the end-to-end dependency paths in the $Slice$.

$$Number_n = \frac{Slice_{back}(n) \cap B}{|B|} \tag{4}$$

$Number_n$ indicates the number of reachable buggy nodes from node n. In the formula, Slice_{back}(i) denotes the backward slice with i as the starting point.

Finally, for each commit, the ground-truth slicing criterion is determined by ranking all nodes according to their scores: (1) the candidate with the highest $Score_c$ is labeled as the *appropriate slicing criterion*, as it maximizes the combined effect of covering buggy nodes and minimizing their dependency distance; (2) the candidate with the lowest $Score_c$ is labeled as the *inappropriate slicing criterion*; and (3) all remaining candidates are discarded. In addition, since the variables and parameters cannot be used independently as the slicing criterion, the appropriate nodes at the substatement level will store the variable type and the belonging statement or method definition as the slicing criterion content.

The score calculation is supposed to include all bug-related nodes, but on the SSLDG for a commit, the buggy nodes may not all be in the backward scope from one node. For example, buggy nodes are distributed in independent methods in the commit[2] and cannot be reached through one program slicing. Therefore, we use the ratio to measure whether the node is appropriate.

The constructed dataset contains 1200 appropriate slicing criteria and 1200 inappropriate criteria by storing the location of

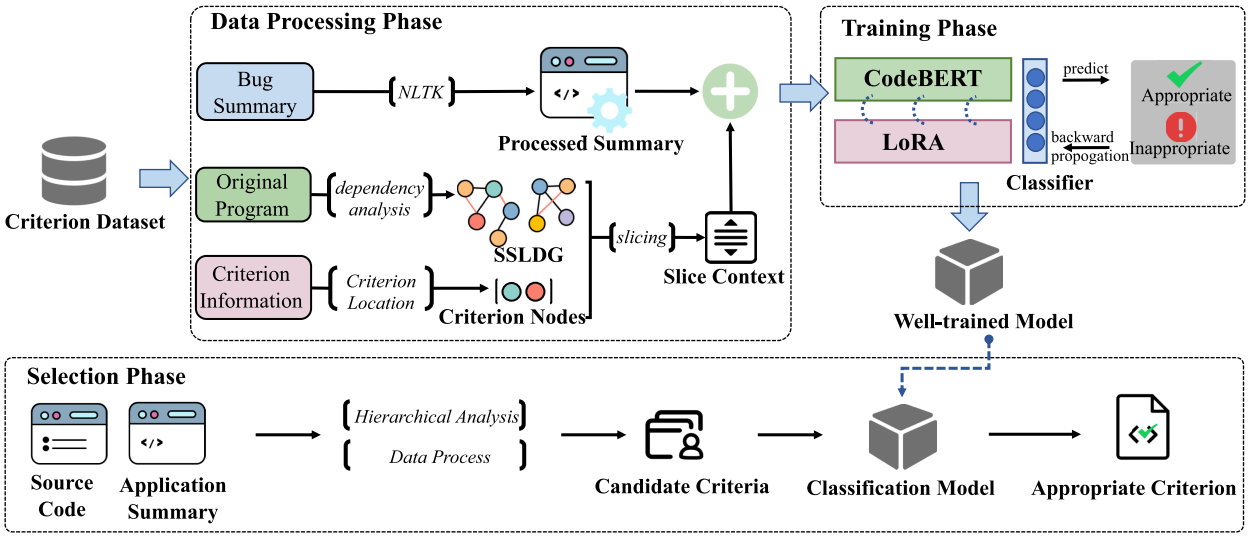

FIGURE 2 | The framework of PSCSelector.

the slicing criterion, the corresponding code content, the source program, the SSLDGs for all commits, and the corresponding commit message. With the dataset, the slicing criterion selection model can extract slicing criterion semantics from the basic code and text.

5 | Our Approach

The implementation of PSCSelector consists of three main phases, as illustrated in Figure 2. In data processing phase, PSCSelector obtains the code context and the corresponding application description of the slicing criterion and uses them as inputs to the model. In training phase, PSCSelector extracts the semantics of the slicing criterion using a lightweight LLM combined with LoRA and performs classification training. In selection phase, the trained model is used for slicing criterion selection according to the original program and application requirements.

5.1 | Data Processing

5.1.1 | Text Processing

The commit message text of one bug-fix commit summarizes the bug and the modification. The processed commit messages are used as textual inputs to guide the model in learning the relationship between the slicing criteria and the bug descriptions. The section utilizes standard text processing techniques, such as stopping word removal, stemming, and tokenization, that help to enhance the quality of the text. The paper employs NLTK [32] to perform natural language preprocessing work. In tokenization, punctuation is removed, and the output is a sequence of words. Stop words such as "an" and "the" are excluded in stop word removal. Stemming is used to eliminate grammatical variations of words and to restore the original word content. After executing natural language preprocessing, the text of commit message becomes more precise and coherent.

5.1.2 | Context Processing

For context extraction of the slicing criterion, PSCSelector obtains omnidirectional slices (forward, backward) from the slicing criterion. The forward slice responds to the impact scope of the slicing criterion, while the backward slice contains the code having a dependency passing on the slicing criterion. Both direction codes are crucial and directly related to the inherent semantics of the criterion. The forward and backward slicing will be carried out simultaneously during slicing, and the specific slicing is shown in Algorithm 1.

Algorithm 1 mainly consists of criterion location (Lines 4–9), backward slicing (Lines 11–15), and forward slicing (Lines 17–21). Before slicing, the algorithm locates the criterion node based on the variable type in the criterion (Line 6). *match* is the traversal process on the dependency graph, distinguishing between forward and backward slicing by dependency direction (arrow direction) and starting point. During both slicings, an efficient two-phase traversal [19] is used to analyze dependency passing. Since interprocedure dependency passes in SSLDG through "*Parameter In*" and "*Parameter Out*" edges, in the backward slicing, the first phase (Line 12) searches reachable caller methods and upper-level nodes like class nodes and restricts the edge "*Parameter Out*" to prevent early entry into the callee methods. The second phase (Line 13) searches reachable nodes of callee methods from the first phase nodes and restricts the edge "*Parameter In*" to prevent re-entry into the caller method for invalid iteration. The forward slicing is opposite to the backward slicing and performs reverse order in direction and phase restriction. With Algorithm 1, PSCSelector obtains the SSLDG nodes in the forward and backward directions related to the slicing criterion.

The algorithm generates a slice that consists of SSLDG nodes, but the large pretrained model requires the text as input. To address this, the paper processes the slices into line granularity:

1) The proposed approach retrieves the corresponding code content based on the node's attribute containing the file

ALGORITHM 1. Slicing from the program slicing criterion Input: SSLDG, Criterion Set $\langle s, v, t \rangle$ named as C

Output: Slice Set Slice_C for C

```
for each Criterion c in C
    Initialize N_c = { }, Slice_c = { }
    // Step 1: Locating the criterion
    for each Node n in SSLDG
        if n = = v and n.type = = t and n ∈ s
            Add n to N_c
        end if
    end for

    // Step 2: Backward slicing
    Match N_1 --e*--> N_c where e ≠ 'Parameter Out'
    Match N_2 --e*--> N_1 where e ≠ 'Parameter In'
    Slice_C = Slice_C ∪ N_2

    // Step 3: Forward slicing
    Match N_c --e*--> N_3 where e ≠ 'Parameter In'
    Match N_3 --e*--> N_4 where e ≠ 'Parameter Out'
    Slice_C = Slice_C ∪ N_4

    Add Slice_c to Slice_C
end for return Slice_C
```

location and line number in the file. The upper level nodes, such as package, class, and method, extract the code statement of the definition without the codes within the structure.

2) Based on the member relationship, the lower substatement level nodes are mapped into the upper nodes. The related statement containing the variable or parameter is extracted into the slice, as a variable cannot represent the type and usage independently without the statements.

3) The order of the lines of code reflects the logic of the code [33]. Therefore, PSCSelector generates the slice following the file and code statement orders.

4) PSCSelector analyzes the variables named by the camel or "_" and splits them according to capital letters and character "_" to restore their original code semantics [34].

With the generated slice, the pretrained model can extract the semantic feature of the slicing criterion.

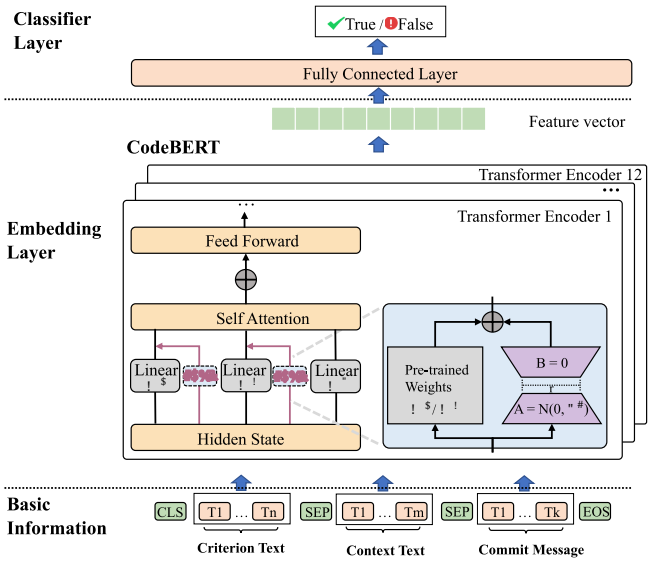

FIGURE 3 | The training workflow of PSCSelector.

5.2 | PSCSelector Fine-Tuning

We adopt CodeBERT as the backbone of PSCSelector because it is an encoder-only pretrained model that fits naturally with code understanding and classification tasks [35]. In addition, CodeBERT as a relatively small model has shown strong and consistent performance across various software engineering applications [36–38], making it a reliable foundation for slicing criterion classification. In the embedding layer of CodeBERT, as illustrated in Figure 3, the input consists of three components: the token sequences of the slicing criterion, the criterion context (the slice), and the commit message content. These sequences are merged into a single sequence and fed into the model in the following format:

$$[CLS]c_1 \ldots c_m[SEP]s_1 \ldots s_n[SEP]i_1 \ldots i_k[EOS]$$

where c_t, s_t, and i_t represent the $t_t h$ token in the criterion code, criterion context, and commit message respectively. The special tokens [CLS], [SEP], and [EOS] follow the standard usage in transformer-based models: [CLS] indicates the beginning of the input sequence, [SEP] serves as a separator between different segments, and [EOS] marks the end of the sequence.

To ensure consistent input length, the token sequence utilizes head-only truncation, with an initial sequence length of 512 tokens. If the sequence length is shorter than the initial length, the input sequence is padded with the special token [pad] to achieve uniformity. Conversely, if the length is higher than the initial length, the truncation is implemented. In our constructed dataset, we have calculated that the number of truncated sequences accounts for only 0.3%.

The basic network structure of the CodeBERT encoder is the 12 transformer encoders. The encoder consists of a bidirectional attention layer and a feedback layer. Each token at the attention layer can attend to context on its left and right. The primary function of the self-attention layer is described as follows:

$$Attention(Q,K,V) = Softmax(\frac{QK^T}{\sqrt{d_k}})$$
$$Q = XW^q, K = XW^k, V = XW^v \tag{5}$$

where X$\in \mathbb{R}^{n\times d}$ is the input tensor and Q, K, V$\in \mathbb{R}^{d\times d}$ are the learned query, key, and value vectors, respectively. They are all essentially linear transformations of X, which are learned to improve the fitting ability of the model by learning matrix W^q, W^k, $W^v \in \mathbb{R}^{d\times d_k}$, d_k is the hidden output dimension of Q and K.

The architecture of the Transformer encoder is complex, and the training overhead is high. However, LoRA is a highly adaptable architecture that can be applied to various models. It can be used to efficiently fine-tune models by imitating the number of parameter changes through a low-rank decomposition. This allows for the indirect training of large models with minimal parameters. Since the self-attention layer is crucial for obtaining code and text semantics in the transform layer, PSCSelector applies LoRA to the self-attention layer. Specifically, PSCSelector replaces the original weight by adding bypass matrices $W^{q\prime}$ and $W^{k\prime}$ for the two weight matrices W^q and W^k, respectively, as shown in Figure 3. The two bypass matrices are constructed by ($B^q \in \mathbb{R}^{d\times r}$, $A^q \in \mathbb{R}^{r\times d_k}$) and ($B^k \in \mathbb{R}^{d\times r}$, $A^k \in \mathbb{R}^{r\times d_k}$), respectively.

$$Q = XW^q + XW^{q'} = XW^q + XB^qA^q$$
$$K = XW^k + XW^{k'} = XW^k + XB^kA^k \tag{6}$$

During model training, only (B^q, A^q) and (B^k, A^k) are updated. After obtaining the slicing criterion feature vector from the optimized pre-trained model, the vector is inputted into a classification layer composed of one fully connected layer, one dropout layer, and one fully connected layer to perform slicing criterion binary classification. The dropout layer randomly zeroes some of the input tensor elements during training to prevent overfitting.

In the training process, we use the binary cross-entropy loss function to train the criterion classifier, and the loss is calculated under the following process:

$$Loss = -\frac{1}{N}\sum_{i=1}^{N}[y_i log(p_i) + (1-y_i)log(1-p_i)] \tag{7}$$

where N is the total number of samples, y_i is the category to which the sample i belongs, and p_i is the predicted value of the sample i, which is a probability value.

5.3 | Criterion Selection

In the selection phase, we apply the trained classification model to select the appropriate program slicing criteria. Specifically, similar to the data processing phase and training phase, the program context of each code statement is obtained by omnidirectional program slicing (Section 5.1.2) and the requirement text is preprocessed through NLTK (Section 5.1.1). Next, the trained large model extracted semantics from the statement context and the processed requirement text (Section 5.2). Finally, the code statement feature is input into the classifier to determine the appropriate slicing criteria.

Since analyzing whether the code statement is an appropriate slicing criterion line by line is a complex process, PSCSelector adopts a hierarchical analysis approach [39, 40] to narrow down the selection scope. Following the hierarchical levels of packages, classes, methods, and statements in SSLDG, PSCSelector first uses package declarations as candidate slicing criteria and performs binary classification. In the appropriate packages, the class declaration statements are then used as candidate slicing criteria. PSCSelector selects the class declarations with the appropriate label. Similarly, in the appropriate methods, PSCSelector finally selects the appropriate slicing criteria at the statement level.

6 | Evaluation

6.1 | Research Questions

RQ 1: How effective are PSCSelector and baseline models for slicing criterion selection? PSCSelector uses the lightweight LLM CodeBERT to encode the program slicing criterion semantics. In addition to CodeBERT, there are five popular code large language models, including CodeT5, CodeT5+, UnixCoder, PLBART, and LLaMA. This question answers the performance of PSCSelector and the five popular LLMs on the program slice criterion selection task.

RQ 2: To what extent do program slicing and LoRA contribute to the PSCSelector? Both program slicing and LoRA are essential components of PSCSelector to improve PSCSelector's ability to learn program slicing criteria. This question explains how much program slicing and LoRA improve PSCSelector.

RQ 3: How effective is PSCSelector in selecting the appropriate slicing criterion for actual bug localization? One of the key contributions of our approach is building the first slicing criterion selection framework, PSCSelector. We aim to show whether PSCSelector is able to effectively select slicing criteria in the actual program slicing application.

6.2 | Evaluation Metrics

In order to measure the effectiveness of PSCSelector, we use the general metrics, including Precision, Recall, ACC, and F1. We also use the following metric, MCC, which is commonly used in previous work [41, 42].

MCC is a statistical score that is commonly used to evaluate binary classifications. It is a reliable indicator of the effectiveness of a prediction and produces a high value only if the prediction is excellent in all items in the confusion matrix. MCC ranges between –1 and 1, where zero indicates that the prediction is random.

$$MCC = \frac{TP\cdot TN - FP\cdot FN}{\sqrt{(TP+FP)\cdot(TP+FN)\cdot(TN+FP)\cdot(TN+FN)}} \tag{8}$$

In slicing-based bug search, a slice should ideally be both concise and informative. A smaller slice indicates higher precision and efficiency [22], whereas containing more buggy statements reflects higher usefulness [43]. To capture this trade-off, we propose the metric *Bug Ratio*, which integrates slice size and the number of buggy statements to measure the overall effectiveness of the selected slicing criterion. A higher *Bug Ratio* value indicates that the criterion is more accurate, as it provides a narrower search scope and helps identify more buggy codes.

$$BugRatio = \frac{Num(b)}{Size(c)} \tag{9}$$

For one bug b, $Num(b)$ represents the number of the buggy code lines and the $Size(c)$ represents the size of the slice from the criterion c.

6.3 | Experiment Setup

6.3.1 | Dataset

Two datasets are chosen for the experiments: the dataset constructed in this paper and the Defects4J dataset. The constructed dataset is used to evaluate PSCSelector's performance in classifying the slicing criteria for RQ 1 and RQ 2, while Defects4J is employed to assess PSCSelector's impact on the downstream task for RQ 3. The data of this paper's dataset are described in Section 4. Defects4J contains records of bug fixes in 14 open-source projects, as shown in Table 2. "Project," "Bug," "Buggy files," and "Changed Loc" are the total number of projects, bugs, files containing bugs, buggy code statements, respectively.

6.3.2 | Baselines

PSCSelector is the first work that explores performing slicing criterion selection with lightweight LLM, and there is no previous work to compare it. Therefore, we select existing popular large language models for the code detection task [44–48] as baselines and evaluates the effectiveness of PSCSelector and these baselines.

CodeT5: CodeT5 [44] is an encoder–decoder model based on the T5 architecture that can support code comprehension and generation tasks. It considers identifier information in code and proposes an identifier-aware pretraining goal that enables the model to capture code semantics better.

CodeT5+: CodeT5+ [45] is a encoder–decoder architecture that supports the pretraining of bimodal models for language and text. It can run in different modes like encoder-only, decoder-only, and encoder–decoder to complete various downstream requirements.

TABLE 2 | The basic information of Defects4J.

[1pt] Project	Bugs	Buggy files	Changed LOC
14	650	1509	5833
[1pt]			

PLBART: PLBART [46] is a sequence-to-sequence model capable of performing various program comprehension and generation tasks. Based on extensive Java and Python functions and the corresponding natural language texts, PLBART is pretrained with autoregressive transformers by denoising autocoding.

UnixCoder: UnixCoder [47] is a cross-modal pretrained code model that considers code comments and abstract syntax trees to enhance code representations in addition to code content. Experimental results show that UnixCoder performs State-of-the-Art (SOTA) in most tasks downstream of the comprehension and generation tasks.

LLaMA-3-8B: LLaMA-3 [48] is optimized for various natural language processing and programming-related tasks. Unlike traditional encoder–decoder architectures, LLaMA-3 focuses on decoder-only pre-training, leveraging large-scale instruction tuning to align model behavior with task requirements.

Qwen-2.5-Coder-7B: Qwen-2.5-Coder [49] is a large language model designed for code understanding and generation tasks. It employs advanced transformer architectures and is fine-tuned on extensive code datasets to enhance its performance in software engineering applications.

6.3.3 | Implementation

To address RQ 1, we combine PSCSelector with each baseline model and compare PSCSelector's performance across different models. Using the constructed slicing criterion dataset, the data are randomly split into training and test sets in an 8:2 ratio. To mitigate potential bias from random splits, we repeat the training process three times and report the mean results. The complete hyperparameter settings used for PSCSelector and all six baseline models are summarized in Table 3. The learning rates

TABLE 3 | Training hyperparameters used for PSCSelector and all baseline models.

Hyperparameter	Symbol	Value
Batch size	B	16
Learning rate search range	lr	$[1\times10^{-5},\ 1\times10^{-4}]$
Learning rate selection	—	Selected by validation F1
Maximum epochs	$E_{\max}$	50
Optimizer	Opt	AdamW
Warmup schedule	—	Linear
LoRA rank	r	8
LoRA α	α	16
LoRA dropout	p_{drop}	0.05
Cutoff length	$L_{\max}$	1024
Early stopping patience	—	7 epochs

for PSCSelector and the six baseline models are tuned within the range of [1e−5, 1e−4], with a step size of 1e−5. For each model, the optimal learning rate is determined based on the best F1 score. The batch size for the six pretrained models is set to 16, and training is conducted for up to 50 epochs, employing an early stopping strategy that halts training if the F1 value does not improve over the last seven epochs. The AdamW optimizer [50] is used with a linear warmup to optimize the models. Additionally, the six baseline models are fine-tuned using LoRA, following the same approach as PSCSelector. The experiment sets the hyperparameters of LoRA following the previous work [17]: Rank is 8, alpha is 16, and dropout is 0.05. Furthermore, we perform instruction fine-tuning on LLaMA-3 and Qwen-2.5-Coder using the following instruction: "Based on the program slicing criterion, its context, and the bug description, determine whether the criterion is most appropriate for performing program slicing to locate the bug. Respond with 'Yes' or 'No'."

To answer RQ 2, the experiments focus on the respective contribution of program slicing and parameter-efficient optimization to PSCSelector. Based on the slicing criterion dataset, the experiment compares PSCSelector with PSCSelector without slicing and PSCSelector without LoRA; the impact of program slicing and LoRA on the classification of the slicing criterion is evaluated based on five experimental metrics.

To answer RQ 3, the experiment applies the well-trained PSCSelector in the actual bug localization application. Utilizing the Defects4J dataset, bug descriptions and original programs serve as inputs to the model. In the experiment, the last bug-triggering statement [7] is used as the slicing criterion, enabling comparison with the criteria selected by PSCSelector. The contribution of program slicing in bug localization is to narrow down the scope of bug search [12–14]. This experiment assesses the effectiveness of PSCSelector by determining whether the selected criteria effectively contribute to bug search.

6.3.4 | Experimental Environment

The PSCSelector implementation involves using Java and Python programming languages. Java is used for dependency extraction and SSLDG construction, while Python is used for model training. PSCSelector is performed on GPU A100-PCIE-40GB, CPU Intel Xeon Processor, and RAM 120GB, running ubuntu20.04 with installed CUDA 11.8.

7 | Experiment Result

7.1 | RQ 1: PSCSelector vs. Baselines

The experiment compares our proposed PSCSelector with five popular pre-trained code models. Table 4 lists the five metric results of our approach and baselines on the dataset constructed in this paper. PSCSelector applies LoRA based on the baseline CodeBERT, while the baseline CodeBERT applies the full-parameter tuning. Specifically, PSCSelector has outperformed the baselines by 4%–21%, 2%–9%, and 11%–88% in terms of Accuracy, F1, and MCC. In addition, UnixCoder and PLBART have obtained the highest recall and precision, respectively. Although PSCSelector slightly underperformed compared to UnixCoder and PLBART in terms of recall and precision, PSCSelector reached the best F1 score, which is the harmonic mean of recall and precision. This indicates that PSCSelector achieves a better balance between recall and precision, making it a more well-rounded model for the task.

TABLE 4 | The result of Experiment 1.

Model	ACC	Precision	Recall	F1	MCC
CodeT5	0.655	0.609	0.866	0.715	0.343
CodeT5+	0.685	0.638	0.857	0.731	0.394
UnixCoder	0.588	0.552	**0.933**	0.694	0.244
PLBART	0.676	**0.659**	0.731	0.693	0.355
LLaMA-3	0.675	0.653	0.857	0.741	0.353
Qwen-2.5-Coder	0.668	0.646	0.846	0.733	0.344
PSCSelector	**0.714**	0.658	0.891	**0.757**	**0.458**

Note: The bolded values in the table indicate the highest value in each column.

When analyzing the results in the program slicing criterion selection, Unixcoder achieves lower results in terms of ACC and F1. This is because the context of the criterion is a static program slice that does not preserve the complete syntax structure, which may negatively affect the Unixcoder based on an abstract syntax tree. To evaluate the effect, we use the harmonic mean F1 as the metric and omit the context of the input of Unixcoder and PSCSelector. The results show that the Unixcoder's F1 of 0.68 is closer to PSCSelector's 0.692; the difference is smaller than the data presented in Table 4, where Unixcoder's F1 is 0.694 and CodeBERT's F1 is 0.757. The results indicate that context without the syntax structure reduces the Unixcoder's performance.

CodeT5 and CodeT5+ achieved relatively better results because they use the Bimodal Dual Generation, which aligns the natural language (NL) and program language (PL), easing the gap between pre-training and fine-tuning. In order to assess the impact of natural language on the models, we removed the commit message from the input. As a result, the F1 scores of the two models were reduced to 0.697 and 0.714, respectively, which are lower than the table results. These scores were still lower than PSCSelector, which had an F1 score of 0.733 under the same input. This identifies that CodeT5 and CodeT5+ are competent in the NL-PL slicing criterion selection task.

PLBART is also a model that focuses on program syntax extraction. Similarly, we compute the model's F1 without context, and the result is 0.681, closer to PSCSelector's 0.692. The difference is smaller than the data presented in Table 4, where PLBART's F1 is 0.693 and CodeBERT's F1 is 0.757. The result indicates that PLBART's performance in the criterion selection task decreases when context lacks syntax structure.

Although LLaMA-3 and Qwen-2.5-Coder are significantly larger than CodeBERT, PSCSelector, which leverages CodeBERT, outperforms both LLaMA-3 and Qwen-2.5-Coder

on the slicing criterion classification task. Specifically, PSCSelector achieves a 6% improvement in ACC and a 2% improvement in F1 compared to LLaMA-3 and a 7% improvement in ACC and 3% improvement in F1 compared to Qwen-2.5-Coder. These gains suggest that CodeBERT's targeted design for code-related tasks allows it to better capture the semantic relationships and structural context of program slicing criteria.

Answer to RQ 1: PSCSelector shows the best performance in the slicing criterion selection task, outperforming the baselines by 4–21%, 2–9%, and 11–88% in terms of Accuracy, F1, and MCC.

7.2 | RQ 2: The Contribution of Program Slicing and LoRA

This experiment analyzes the contribution of two components of PSCSelector: the LoRA for parameter fine-tuning and the program slicing for semantic extraction. For LoRA analysis, we analyze the parameter scale and time in addition to the five metrics, and the results are shown in Table 5. For program slicing analysis, we analyze the two results of PSCSelector without the slice and the context. PSCSelector without slice replaces the slices with the block content where the slicing criterion belongs, and the experiment results are shown in Table 6.

In Table 5, the se number of parameters of the model, *GPU Memory* is the size of the used GPU memory, and *Epoch Time* and *Testing Time* are the average training time and testing time of the model, respectively. Based on the experimental data, we can observe that LoRA activates a parameter size of 591,361 during the training processing. This parameter size accounts for only 0.47% of the original CodeBERT's 125,532,674. Moreover, LoRA reduces the average epoch time by 2.2 s and the testing time by 0.2 s. It also reduces the original 2403 Mb GPU memory usage by 21.04%. At the same time, the five metrics also clearly show that LoRA improves CodeBERT's performance in the task of classifying program slicing criterion. LoRA improves CodeBERT's F1 value by 1.88%, ACC value by 4.85%, and precision by 5.79% under the reduction of Recall by 3.57%. Based on the results, LoRA improves PSCSelector's performance while reducing time and GPU memory usage. The potential reason is that LoRA uses trainable low-rank matrices during weight updating without tuning full parameters. The experiment demonstrates that LoRA is helpful for PSCSelector in learning the slicing criterion.

In Table 6, PSCSelector$_{Block}$ learns the criterion using the block as the context of the criterion instead of the slice, where the block is a code set containing codes in the same definition domain as the criterion [22]. PSCSelector$_{noContext}$ learns the criterion without the context. Table 6 shows that PSCSelector containing the slice gets the highest results for all five metrics, and PSCSelector$_{Block}$ is relatively low for all. The reason for the difference is the semantics of the two contexts. The slice contains codes that have dependencies on the criterion, reflecting the semantics of the criterion itself. In contrast, the blocks may contain noisy codes unrelated to the criterion. Although noisy codes and the criterion belong to the same definition domain and are adjacent in the text, these noisy codes have no dependencies for the criterion. The noisy codes may affect extracting the criterion's features, leading to the model's low results for each metric. In addition, according to the data of PSCSelector$_{noContext}$, context information can extend the semantics of the criterion from itself and is helpful for PSCSelector to learn the slicing criterion. Therefore, the context complements the semantics of the criterion, and the context obtained by the slicing method describes the criterion more accurately.

Answer to RQ 2: Both LoRA and program slicing are critical processes in PSCSelector. LoRA reduces the training parameters and GPU memory usage by 99.53% and 21.04%, respectively, while concurrently improving the F1 value of PSCSelector by 1.88%. The program slice can better represent the semantics of the criterion and improve the PSCSelector for the learning criterion, with an improving F1 value of 17%.

TABLE 5 | The contribution of LoRA.

Model	ACC	Precision	Recall	F1	MCC	Parameters	Epoch time	Testing time	GPU memory
PSCSelector	**0.714**	**0.658**	0.891	**0.757**	**0.458**	591361	24.3s	0.9s	1897.5Mb
PSCSelector$_{noLoRA}$	0.681	0.622	**0.924**	0.743	0.414	125532674	26.5s	1.1s	2403.0Mb

Note: The bolded values in the table indicate the highest value in each column.

TABLE 6 | The contribution of program slicing.

Model	ACC	Precision	Recall	F1	MCC
PSCSelector	**0.714**	**0.658**	**0.891**	**0.757**	**0.458**
PSCSelector$_{Block}$	0.597	0.575	0.739	0.647	0.202
PSCSelector$_{noContext}$	0.622	0.583	0.857	0.694	0.276

Note: The bolded values in the table indicate the highest value in each column.

7.3 | RQ 3: The Effectiveness of PSCSelector in Bug Localization

7.3.1 | Results

The experiment evaluates the well-fine-tuned PSCSelector in actual bug localization and compares the selected criteria of PSCSelector with the bug-triggering statement. Based on the Defects4J dataset, PSCSelector employs a hierarchical analysis method to analyze the code for each bug. PSCSelector takes the input from the code, program slice, and the bug description to select the appropriate slicing criteria. With PSCSelector-selected criteria, program slicing is performed to search for the bug. Since 117 bug-related programs contain only added code without buggy codes, namely, modified or deleted codes [51], Table 7 presents the experimental results of the PSCSelector on the 533 bugs.

In Table 7, *Slice size* denotes the average slice size of the slicing criterion in searching 533 bugs. The slice size is measured in code lines. *Bug Ratio* denotes average value of the Bug Ratios of the slicing criteria for searching 533 bugs. Since Defects4J is typically for single-fault scenarios, searching all buggy codes of one bug means the bug is searched. *Success* denotes the number of bugs successfully searched using the slicing criterion.

As shown in Table 7, utilizing PSCSelector-selected slicing criteria has resulted in a significant reduction in slice size, with an average decrease of 682 lines, representing a 74% reduction compared to the bug-triggering statements. Moreover, the average Bug Ratio of PSCSelector-selected slicing criteria is 0.043, improving the bug-triggering statements by 291%. From the above statistics, PSCSelector-selected slicing criteria can significantly reduce the search scope and effectively help in searching the bug.

TABLE 7 | The experiment result for bug search.

Criterion	Slice size	Bug Ratio	Success Bug
PSCSelector-selected criteria	236	0.043	395
Bug-triggering statement	918	0.011	421

Commit is f76efe5f4ef36fadc677c94269927076f2f42eb9
src/main/java/org/apache/commons/math/distribution/PoissonDistributionImpl.java

```
79   public PoissonDistributionImpl(double p) {
80     this(p, new NormalDistributionImpl());
92   public PoissonDistributionImpl(double p, double epsilon, int maxIterations){
93     setMean(p);
105  public PoissonDistributionImpl(double p, double epsilon) {
106    setMean(p);
118  public PoissonDistributionImpl(double p, int maxIterations) {
119    setMean(p);
135  public PoissonDistributionImpl(double p, NormalDistribution z) {
136    super();
158  public void setMean(double p) {
159    setNormalAndMeanInternal(normal, p);
300  public void setNormal(NormalDistribution value) {
301    setNormalAndMeanInternal(value, mean);
302  }
```

FIGURE 4 | The example of scattered buggy code.

Although the PSCSelector-selected slicing criteria search the slightly lower bugs than bug-triggering statements by 26, PSCSelector-selected slicing criteria can search the most (74%, 395/533) of the bugs. In the unsuccessfully searched bugs, we have observed that the buggy codes are distributed across separate functions, and no dependency is passing between them. For the example, in Figure 4, the buggy codes are distributed in the overloading function (Lines 79-136) and the independent functions (Lines 158 and 300). These buggy codes are unable to be searched by program slicing using bug-triggering statements or selected criteria. For the 26 bugs where PSCSelector fails to search for bugs while the bug-triggering statement is successful, we have noticed that the average length of the forward dependency path for these buggy codes is less than 4. They have fewer dependency passes with other codes in the project. As a result, the buggy codes become challenging to search by the selected criteria.

To further analyze PSCSelector's effectiveness, we calculate the number of slicing criteria selected by PSCSelector and the detailed size of the slices generated from the selected criteria in the 395 successfully searched bugs.

7.3.2 | The Analysis of the Number of PSCSelector's Slicing Criteria

Figure 5 shows the number of slicing criteria selected by PSCSelector for the 395 bugs. The x-axis represents 395 bugs; the y-axis is the number of selected criteria. In one bug, the selection result of PSCSelector is shown as the pink scatter, while the blue scatter represents the number of criteria that can successfully search all buggy codes. Results are sorted by the number of PSCSelector-selected slicing criteria. On average, the number of slicing criteria selected by PSCSelector is 11, and 6 of them can successfully search the bug. Furthermore, in 90% of the bugs, PSCSelector selects less than 20 slicing criteria for bug localization. Based on the statistics, we can conclude that PSCSelector can effectively generate a small-scale number of slicing criteria with an average number of 11 for the bug localization.

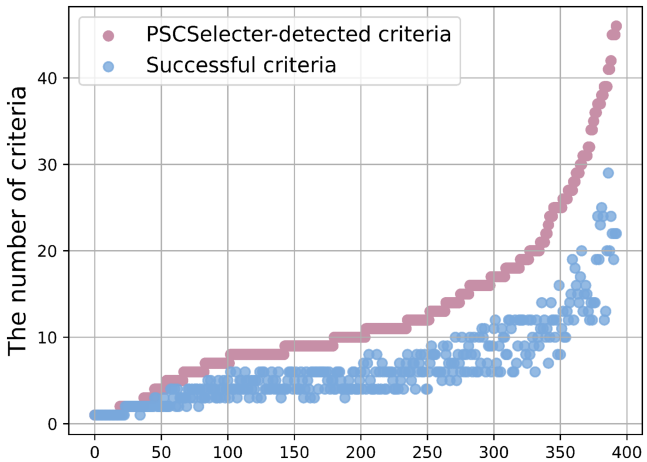

FIGURE 5 | The count of slicing criteria selected by PSCSelector for successful bug search.

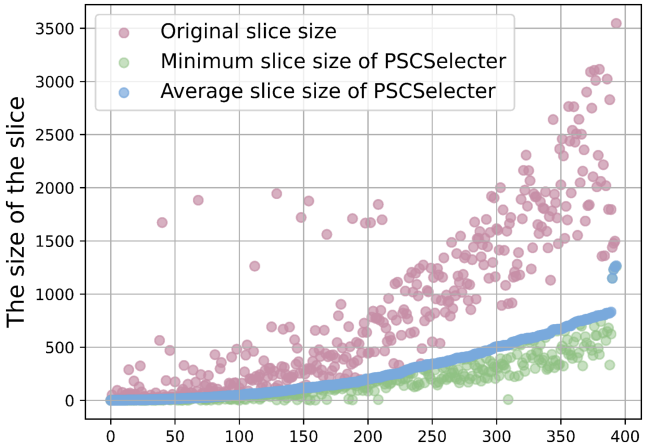

FIGURE 6 | The Count of slice size from selected criteria for successful bug search.

7.3.3 | The Detailed Analysis of the Slice Size From PSCSelector-Selected Criteria

The effectiveness of PSCSelector is not limited to the number of searched bugs; it also focuses on the search scope using the selected slicing criterion. We further measure the size of the slices generated by PSCSelector, which successfully search for bugs, and compare it to the size of the original slices with the bug-triggering statement as the criterion. The experiment result is shown in Figure 6. The x-axis is 395 bugs, and the y-axis is the size of the slice. The blue scatter represents the average size of the backward program slice with the PSCSelector-generated slicing criteria; the pink scatter is the size of the original slice with the bug-triggering statement as the slicing criterion; the green scatter shows the minimum slice size generated by the slicing criterion from PSCSelector, which has the ability to search the bug. Results are sorted by the average slice size of the PSCSelector-selected criteria. Based on the experiment result, we make the following statistics. In most (99%, 390/395) bug searches, PSCSelector-selected slicing criteria generate smaller slices, while in five bugs, both criteria have the same slice size. On average, the PSCSelector-generated slicing criterion generates 276 lines of slice and reduces the original slice size by 69%, which is 903 lines of slice. Among 395 bugs, the PSCSelector-selected criterion can reduce the original slices by a maximum of 99.8%. From the statistics, we can conclude that PSCSelector can effectively narrow down the search scope for bug localization.

Answer to RQ 3: PSCSelector can effectively select the slicing criteria for bug localization. With PSCSelector, 74% of bugs in the Defects4J can be effectively searched. On average, the search scope is reduced by 69%, and the Bug Ratio is improved by 291% compared to using the bug-triggering statement as the slicing criterion.

8 | Threats to Validity

Internal validity. Internal validity in our experiments relates to two factors. The first is the labeling of the slicing criterion. In the study, the slicing criterion labeling in the dataset only considers the factors for the distance and the number. Therefore, there may be other factors affecting the criterion labeling. In order to reduce the influence of other factors, we tried our best to construct accurate slicing criterion samples to be checked by three experienced developers. In addition, the fine-tuning of the benchmark model may affect the model's performance on the constructed dataset. We try to embed the slicing criterion information according to the recommended learning rate from their respective papers.

External validity. The main external validity is the generalizability of the PSCSelector. Since the SSLDG construction is based on the Java program, and the experiment results may not be reproducible in other popular languages like C/C++. However, the methods in PSCSelector are language-agnostic, and practitioners can fine-tune PSCSelector by building their own system dependency graphs and datasets on other languages.

9 | Related Work

PSCSelector is the framework using the pre-trained model to select slicing criteria during program slicing. The related work of the paper focuses on the application of program slicing criterion selection and the application of pretrained models in program slicing.

9.1 | Application of Program Slicing Criterion Selection

With program slicing applications, the selection of slicing criterion is widely used in software engineering downstream tasks. For example, in change impact analysis, Ufuktepe [52] uses all changed functions and their parameters as the slicing criteria and performs forward program slicing on the static call graph to obtain the impact range of the changed code. In defect detection, Alqadi [5] uses each program segment in the project as a slicing criterion. He obtains slices based on data and control dependencies and then predicts whether the code is defective using slice metrics. In vulnerability detection, Salimi [53] uses statements with execution errors and their variables as the slicing criteria to extract the set of statements associated with the errors. Cheng [54] searches for vulnerability-fixing nodes based on the procedure dependency graph using all API calls as the slicing criterion. Cao [4] detects the vulnerability using all the API calls and pointer variables as the slicing criterion to obtain a prediction of whether the code is vulnerable or not through learning the slices. Similarly, Zou [55] uses all API calls as the slicing criteria and learns the slice features to detect whether the project contains vulnerabilities. In code bad smell detection, Alomari [56] uses all variables as slicing criteria and computes the corresponding slices using the SRCSLICE tool [57] to convert code clone detection to slice clone detection.

In the above applications of program slicing, the selection of slicing criteria is usually untargeted. All statements of a particular form are chosen as the slicing criteria, which may lead to the generation of large slices. However, the framework PSCSelector proposed in this paper can achieve a targeted selection of slicing criteria and reduce the size of slicing analysis in specific applications.

9.2 | Application of Pretrained Models in Program Slicing

Currently, many researchers attempt to improve program slicing methods using the large language model. For example, Yadavally [2] proposed NS-Slicer, a new learning-based approach to predicting static slices in incomplete code by learning the dependencies between the slicing criterion and other statements, solving the traditional problem of not being able to generate slices due to missing dependencies. Meanwhile, for dynamic slicing, Yadavally [43] also proposed ND-Slicer, which uses execution-aware training to learn dynamic program dependencies, similar to predictive slicing NS-Slicer, but learns dynamic data dependencies and control dependencies between the criteria and other statements. Wang [58] proposed LLMDFA, an LLM-powered framework for data flow analysis. Without compilation infrastructure, LLMDFA can analyze the flow dependency passing of arbitrary code fragments. Zhang [59] proposed the CodeBERT-slicing method. Instead of relying on complete traversal, Zhang uses pretrained CodeBERT to classify the paths in the control flow graph and generate slices based on the filtered paths.

The above methods aim to enhance program slicing by replacing intermediate representations of programs with training the large language models. These methods either replace or optimize the dependency analysis but do not address the selection of the slicing criterion, which is an essential step at the beginning of the slicing. PSCSelector in the paper learns to give the slicing criterion through a pretrained model that makes the overall program slicing intelligent.

10 | Conclusion and Future Work

In this paper, we propose PSCSelector, the pretrained model-based framework to select appropriate slicing criteria in the program slicing application. PSCSelector first constructs the fine-grained sub-statement level system dependency graph and performs program slicing to extract the slicing context. After program slicing, PSCSelector employs the parameter-efficient fine-tuned CodeBERT to jointly embed both the criterion context and application text descriptions to learn appropriate slicing criterion patterns. We have applied PSCSelector in bug search and demonstrate that PSCSelector effectively searches 74% bugs by reducing 69% slice size. In addition, a criterion dataset for bug search is constructed to assist future research on slicing criterion detection. PSCSelector and the dataset are stored in the repository.[3]

In the future, we intend to explore the effectiveness of PSCSelector in other applications, such as change impact analysis. In addition, we are interested in constructing criteria datasets tailored to these specific applications.

Conflicts of Interest

The authors declare no conflicts of interest.

Data Availability Statement

The data that support the findings of this study are openly available in PSCSelecter-8C31 at https://anonymous.4open.science/r/PSCSelecter-8C31.

Endnotes

[1] The commit id is "c338c8aca40d43efda580057f6d41ba228e26ec1".

[2] The commit id is f76efe5f4ef36fadc677c94269927076f2f42eb9.

[3] https://anonymous.4open.science/r/PSCSelecter-8C31.

References

1. M. Weiser, "Program Slicing," *IEEE Transactions on Software Engineering* 10, no. 4 (1984): 352–357.

2. A. Yadavally, Y. Li, S. Wang, and T. N. Nguyen, "A Learning-Based Approach to Static Program Slicing," in *ACM SIGPLAN International Conference on Object-Oriented Programming Systems, Languages, and Applications (OOPSLA)* (Association for Computing Machinery, 2024), To Appear.

3. D. Binkley, N. Gold, M. Harman, S. Islam, J. Krinke, and S. Yoo, "ORBS: Language-Independent Program Slicing," in *Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering (FSE)* (Association for Computing Machinery, 2014), 109120.

4. S. Cao, X. Sun, L. Bo, R. Wu, B. Li, and C. Tao, "MVD: Memory-Related Vulnerability Detection Based on Flow-Sensitive Graph Neural Networks," in *2022 IEEE/ACM 44th International Conference on Software Engineering (ICSE)* (Association for Computing Machinery, 2022), 1456–1468.

5. B. S. Alqadi and J. I. Maletic, "Integration of Program Slicing With Cognitive Complexity for Defect Prediction," in *2020 IEEE International Conference on Software Maintenance and Evolution (ICSME)* (IEEE, 2020), 839–843.

6. J. Xuan and M. Monperrus, "Test Case Purification for Improving Fault Localization," in *Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering (FSE)* (Association for Computing Machinery, 2014), 52–63.

7. E. Soremekun, L. Kirschner, M. Böhme, and A. Zeller, "Locating Faults With Program Slicing: An Empirical Analysis," *Empirical Software Engineering* 26, no. 3 (2021): 1–45.

8. X. Cheng, X. Nie, N. Li, H. Wang, Z. Zheng, and Y. Sui, "How About Bug-Triggering Paths? - Understanding and Characterizing Learning-Based Vulnerability Detectors," *IEEE Transactions on Dependable and Secure Computing* 21, no. 2 (2024): 542–558.

9. Z. Liu, Z. Tang, J. Zhang, X. Xia, and X. Yang, "Pre-Training by Predicting Program Dependencies for Vulnerability Analysis Tasks," in *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering (ICSE)* (IEEE, 2024).

10. Z. Feng, D. Guo, D. Tang, et al., "CodeBERT: A Pre-Trained Model for Programming and Natural Languages," in *Findings of the Association for Computational Linguistics: EMNLP* (2020), 1536–1547.

11. X. Zhou, D. Han, and D. Lo, "Assessing Generalizability of CodeBERT," in *2021 IEEE International Conference on Software Maintenance and Evolution (ICSME)* (IEEE, 2021), 425–436.

12. T. Dao, L. Zhang, and N. Meng, "How Does Execution Information Help With Information-Retrieval Based Bug Localization?," in *2017 IEEE/ACM 25th International Conference on Program Comprehension (ICPC)* (IEEE, 2017), 241–250.

13. B. Liang, P. Bian, Y. Zhang, W. Shi, W. You, and Y. Cai, "AntMiner: Mining More Bugs by Reducing Noise Interference," in *2016 IEEE/ACM 38th International Conference on Software Engineering (ICSE)* (Association for Computing Machinery, 2016), 333–344.

14. Y. Lin, J. Sun, L. Tran, G. Bai, H. Wang, and J. Dong, "Break the Dead End of Dynamic Slicing: Localizing Data and Control Omission Bug," in *2018 33rd IEEE/ACM International Conference on Automated Software Engineering (ASE)* (Association for Computing Machinery, 2018), 509–519.

15. X. Xie, T. Y. Chen, F. C. Kuo, and B. Xu, "A Theoretical Analysis of the Risk Evaluation Formulas for Spectrum-Based Fault Localization," *ACM Transactions on Software Engineering and Methodology* 22, no. 4 (2013): 31.

16. Y. Huang, J. Wu, Y. Feng, Z. Chen, and Z. Zhao, "An Empirical Study on Clustering for Isolating Bugs in Fault Localization," in *2013 IEEE International Symposium on Software Reliability Engineering Workshops (ISSREW)* (IEEE, 2013), 138–143.

17. E. J. Hu, Y. Shen, P. Wallis, et al., "LoRA: Low-Rank Adaptation of Large Language Models," in *International Conference on Learning Representations (ICLR)* (OpenReview.net, 2022).

18. W. E. Wong, R. Gao, Y. Li, R. Abreu, and F. Wotawa, "A Survey on Software Fault Localization," *IEEE Transactions on Software Engineering* 42, no. 8 (2016): 707–740.

19. S. Horwitz, T. Reps, and D. Binkley, "Interprocedural Slicing Using Dependence Graphs," *ACM Transactions on Programming Languages and Systems* 12, no. 1 (1990): 35.

20. K. J. Ottenstein and L. M. Ottenstein, "The Program Dependence Graph in a Software Development Environment," *SIGSOFT Softw. Eng. Notes* 9, no. 3 (1984): 177184, https://doi.org/10.1145/390010.808263.

21. N. Walkinshaw, M. Roper, and M. Wood, "The Java System Dependence Graph," in *Proceedings Third IEEE International Workshop on Source Code Analysis and Manipulation* (IEEE, 2003), 55–64.

22. L. Wang, B. Li, and X. Kong, "Type Slicing: An Accurate Object-Oriented Slicing Based on Sub-Statement Level Dependence Graph," *Information and Software Technology* 127 (2020): 106369.

23. B. Li, X. Fan, J. Pang, and J. Zhao, "A Model for Slicing JAVA Programs Hierarchically," *Journal of Computer Science and Technology* 19, no. 6 (2004): 848858, https://doi.org/10.1007/BF02973448.

24. J. Krinke, "Barrier Slicing and Chopping," in *2013 IEEE 13th International Working Conference on Source Code Analysis and Manipulation (SCAM)* (IEEE, 2003), 81.

25. B. Korel and J. Laski, "Dynamic Program Slicing," *Information Processing Letters* 29, no. 3 (1988): 155–163.

26. N. Ding, Y. Qin, G. Yang, F. Wei, and Z. Yang, "Parameter-Efficient Fine-Tuning of Large-Scale Pre-Trained Language Models," *Nature Machine Intelligence* 5, no. 3 (2023): 220–235.

27. CCS Balne, S Bhaduri, T Roy, V Jain, and A Chadha, "Parameter Efficient Fine Tuning: A Comprehensive Analysis Across Applications," *arXiv*. 2024;2404.13506.

28. Z Han, C Gao, J Liu, J Zhang, and SQ Zhang, "Parameter-Efficient Fine-Tuning for Large Models: A Comprehensive Survey," *arXiv*. 2024;2403.14608.

29. M. Sintaha, N. Nashid, and A. Mesbah, "Katana: Dual Slicing Based Context for Learning Bug Fixes," *ACM Transactions on Software Engineering and Methodology* 32, no. 4 (2023): 100, https://doi.org/10.1145/3579640.

30. Z. Jiang, J. Chen, D. Rao, and G. Ye, "Leveraging Context-Aware Prompting for Commit Message Generation," in *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing* (Association for Computational Linguistics, 2024), 13522–13540.

31. N. Nashid and M. Sintaha, "Mesbah a. Embedding Context as Code Dependencies for Neural Program Repair," in *2023 IEEE Conference on Software Testing, Verification and Validation (ICST)* (IEEE, 2023), 95–106.

32. S. Bird and E. Loper, "NLTK: The Natural Language Toolkit," in *Proceedings of the ACL Interactive Poster and Demonstration Sessions* (Association for Computational Linguistics, 2004), 214–217.

33. J. Zhang, X. Wang, H. Zhang, H. Sun, K. Wang, and X. Liu, "A Novel Neural Source Code Representation Based on Abstract Syntax Tree," in *2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE)*, ed. 794 (Association for Computing Machinery, 2019), 783.

34. S. Wang, X. Wang, K. Sun, S. Jajodia, H. Wang, and Q. Li, "GraphSPD: Graph-Based Security Patch Detection With Enriched Code Semantics," in *2023 IEEE Symposium on Security and Privacy (SP)* (IEEE, 2023), 2409–2426.

35. J. Jiang, F. Wang, J. Shen, S. Kim, and S. Kim, "A Survey on Large Language Models for Code Generation," *ACM Transactions on Software Engineering and Methodology* (2025): 58, https://doi.org/10.1145/3747588.

36. Y. Zhao, L. Gong, Y. Yu, Z. Huang, and M. Wei, "An Empirical Study of Best Practices for Code Pre-Trained Models on Software Engineering Classification Tasks," *Expert Systems With Applications* 272 (2025): 126762.

37. Y. Cai, A. Yadavally, A. Mishra, G. Montejo, and T. Nguyen, "Programming Assistant for Exception Handling With CodeBERT," in *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering (ICSE)* (IEEE, 2024).

38. Y. Xiao, X. Zuo, X. Lu, J. S. Dong, X. Cao, and I. Beschastnikh, "Promises and Perils of Using Transformer-Based Models for SE Research," *Neural Networks* 184 (2025): 107067.

39. Z. Zhong, C. T. Li, and J. Pang, "Multi-Grained Semantics-Aware Graph Neural Networks," *IEEE Transactions on Knowledge and Data Engineering* 35, no. 7 (2023): 7251–7262.

40. T. Wang and A. Roychoudhury, "Hierarchical Dynamic Slicing," in *International Symposium on Software Testing and Analysis (ISSTA)* (Association for Computing Machinery, 2007), 228238.

41. J. Lan, L. Gong, J. Zhang, and H. Zhang, "BTLink: Automatic Link Recovery Between Issues and Commits Based on Pre-Trained BERT Model," *Empirical Softw. Engg* 28, no. 4 (2023): 1–55.

42. D. Chicco and G. Jurman, "The Advantages of the Matthews Correlation Coefficient (MCC) Over F1 Score and Accuracy in Binary Classification Evaluation," *BMC Genomics* 28, no. 21 (2020): 1–13.

43. A. Yadavally, Y. Li, and T. N. Nguyen, "Predictive Program Slicing via Execution Knowledge-Guided Dynamic Dependence Learning," *Proceedings of the ACM on Software Engineering* 1, no. FSE (2024): 271–292.

44. Y. Wang, W. Wang, S. Joty, and S. C. Hoi, "CodeT5: Identifier-Aware Unified Pre-Trained Encoder-Decoder Models for Code Understanding and Generation," in *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing (EMNLP)* (Association for Computational Linguistics, 2021), 8696–8708.

45. Y. Wang, H. Le, A. D. Gotmare, N. D. Q. Bui, J. Li, and S. C. H. Hoi, "CodeT5+: Open Code Large Language Models for Code Understanding and Generation," in *Conference on Empirical Methods in Natural Language Processing (EMNLP)* (Association for Computational Linguistics, 2023), 1069–1088.

46. W. Ahmad, S. Chakraborty, B. Ray, and K. W. Chang, "Unified Pre-Training for Program Understanding and Generation," in *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT)* (Association for Computational Linguistics, 2021), 2655–2668.

47. D. Guo, S. Lu, N. Duan, Y. Wang, M. Zhou, and J. Yin, "UniXcoder: Unified Cross-Modal Pre-Training for Code Representation," in *Annual Meeting of the Association for Computational Linguistics (ACL)* (Association for Computational Linguistics, 2022).

48. T Hugo, L Thibaut, I Gautier, et al., "LLaMA: Open and Efficient Foundation Language Models," *arXiv*. 2023.

49. B Hui, J Yang, Z Cui, et al., "Qwen2.5-Coder Technical Report," *arXiv*. 2024.

50. I. Loshchilov and F. Hutter, "Decoupled Weight Decay Regularization," in *International Conference on Learning Representations (ICLR)* (OpenReview.net, 2019).

51. M. Tufano, C. Watson, G. Bavota, M. Di Penta, M. White, and D. Poshyvanyk, "Learning How to Mutate Source Code From Bug-Fixes," in *IEEE International Conference on Software Maintenance and Evolution (ICSME)* (IEEE, 2019), 301–312.

52. E. Ufuktepe, T. Tuglular, and K. Palaniappan, "Tracking Code Bug Fix Ripple Effects Based on Change Patterns Using Markov Chain Models," *IEEE Transactions on Reliability* 71, no. 2 (2022): 1141–1156.

53. S. Salimi and M. Kharrazi, "VulSlicer: Vulnerability Detection Through Code Slicing," *Journal of Systems and Software* 193, no. C (2022): 1–12.

54. X. Cheng, H. Wang, J. Hua, G. Xu, and Y. Sui, "DeepWukong: Statically Detecting Software Vulnerabilities Using Deep Graph Neural Network," *ACM Transactions on Software Engineering and Methodology* 30, no. 3 (2021): 1–33.

55. D. Zou, S. Wang, S. Xu, Z. Li, and H. Jin, "VulDeePecker: A Deep Learning-Based System for Multiclass Vulnerability Detection," *IEEE Transactions on Dependable and Secure Computing* 18, no. 5 (2021): 2224–2236.

56. H. W. Alomari and M. Stephan, "Clone Detection Through srcClone: A Program Slicing Based Approach," *Journal of Systems and Software* 184, no. C (2022): 1–21.

57. C. D. Newman, T. Sage, M. L. Collard, H. W. Alomari, and J. I. Maletic, "srcSlice: A Tool for Efficient Static Forward Slicing," in *2016 IEEE/ACM 38th International Conference on Software Engineering Companion (ICSE-C)* (Association for Computing Machinery, 2016), 621–624.

58. C Wang, W Zhang, Z Su, X Xu, X Xie, and X Zhang, "When Dataflow Analysis Meets Large Language Models," ArXiv,2024;abs/2402.10754.

59. J. Zhang, Z. Liu, X. Hu, X. Xia, and S. Li, "Vulnerability Detection by Learning From Syntax-Based Execution Paths of Code," *IEEE Transactions on Software Engineering* 49, no. 8 (2023): 4196–4212.